

CourseCompass Web Application Project Brief

Supervisor: Binglan Han (b.han1@massey.ac.nz)

1. Overview and Aim

Students often choose courses using scattered information from handbooks, friends, forums, emails, and social media. It can be difficult to compare workload, prerequisites, assessment types, timetable fit, and peer experiences in one place. This project requires each group to design, implement, test, deploy, and present CourseCompass: a complete web application that helps students review courses, plan future enrolments, compare options, and receive personalised or AI-enhanced course planning support while demonstrating sound software engineering practice. The system should support planning decisions but must not replace official academic advice.

2. Target Users

- Students: create profiles, browse courses, read and submit reviews, save course plans, compare options, and track degree progress informally.
- Course contributors or moderators: add course information, moderate reviews, and maintain course-related guidance where permitted.
- Administrators: manage users, courses, reviews, reports, categories, and system data.

3. Core Functional Requirements

Function	Required features
User accounts and profiles	Register, log in, manage accounts, and maintain profiles with programme, year, completed courses, interests, goals, and planning preferences. Support student, moderator, and administrator roles.
Course directory	Provide course records with code, title, description, prerequisites, semester, level, credits, assessment types, workload estimate, and official link.
Reviews and ratings	Students submit structured reviews including workload, difficulty, teaching experience, assessment style, usefulness, and comments. Support moderation and reporting.
Course planning tools	Students save potential courses, build semester plans, check simple prerequisite warnings, and compare options through a dashboard or list view.
Search and filtering	Search and filter courses by keyword, code, subject, level, semester, workload, assessment type, rating, or prerequisite status.
Notifications and admin dashboard	Notify users about saved course updates, review moderation, and reported content. Admin dashboard includes management tools and statistics on courses, reviews, plans, and reports.

4. AI-Enhanced Requirement

Each group must implement at least one AI-related or intelligent function and explain it clearly in the final report, including the data, algorithm, model, API, or library used.

- Course recommendation: suggest courses based on programme, interests, completed courses, saved courses, and semantic similarity.
- Smart search: support natural-language queries such as “courses with practical programming projects and low exam weight”.
- AI review summary: summarise multiple student reviews into key themes while avoiding misleading claims.
- Planning assistant: generate a draft semester plan from constraints, clearly requiring student verification against official rules.
- Review moderation support: flag inappropriate, duplicate, or low-quality reviews for administrator review.

5. Technical, Database, Interface, and Quality Requirements

- Technical approach: use an appropriate front-end and back-end technology stack; justify the choices in project proposal and/or reports.
- Database: design a relational or document database and document it with an ERD. Demonstrate keys, relationships, constraints, and validation.
- Interface: include home page, login/registration, student dashboard, course list/detail pages, review submission page, course comparison page, semester planner, saved courses page, admin dashboard, and profile page.
- Quality: address usability, security, reliability, maintainability, scalability, accessibility, responsible AI, and privacy. Passwords must not be stored in plain text; if an external AI API is used, explain what data is sent and how risks are managed.

6. Deliverables and Final Report

Area	Requirements
Deliverables across four assessments	Working web application and source repository; database design, setup scripts, and implementation evidence; reports, README/user guide, and testing evidence; video demonstration or live presentation; individual contribution statement.
Final report content	Background, objectives, requirements, architecture, ERD, UI design, implementation details, AI function explanation, testing results, security and ethics, project management, limitations, future work, conclusion, and references.
Academic expectations	Use clear academic writing and cite external tools, libraries, frameworks, datasets, APIs, and AI models where relevant.

7. Group Roles, Assessment Focus, and Extensions

Suggested roles are project manager/requirements lead, front-end developer, back-end developer, and database/AI developer. Roles support coordination, but all members must understand the full system and contribute to development, reporting, and presentation.

Assessment will consider functional completeness, correctness, architecture, database design, code quality, maintainability, usability, AI implementation, testing, report quality, presentation, and teamwork evidence. Possible extensions include degree progress visualisation, timetable conflict checks, official link integration, AI review summaries, recommendation explanations, peer discussion threads, analytics dashboards, mobile responsiveness, and planning badges.

8. Expected Outcome

By the end of the project, each group should deliver a functional web application that addresses a realistic student course planning problem and shows the ability to build a complete web-based system using modern development practices, including a meaningful and responsibly explained intelligent or AI-enhanced feature.